SPANS-06: Security Analysis with Spans

> **Series:** SPANS — Distributed Tracing and Spans | **Notebook:** 6 of 8 | **Created:** December 2025 | **Last Updated:** 10/28/2026

Protecting Distributed Traces and Ensuring Compliance

This notebook demonstrates how to use span data for security analysis, audit for sensitive data exposure, and ensure compliance with regulations.

Table of Contents

1. Understanding Sensitive Data in Spans
2. PII Audit Queries
3. HTTP Status Code Security Analysis
4. Authentication Failure Detection
5. Anomalous Traffic Patterns
6. Sensitive Endpoint Monitoring
7. OpenPipeline for Data Masking
8. Compliance Considerations
9. Security Audit Queries
10. Security Checklist

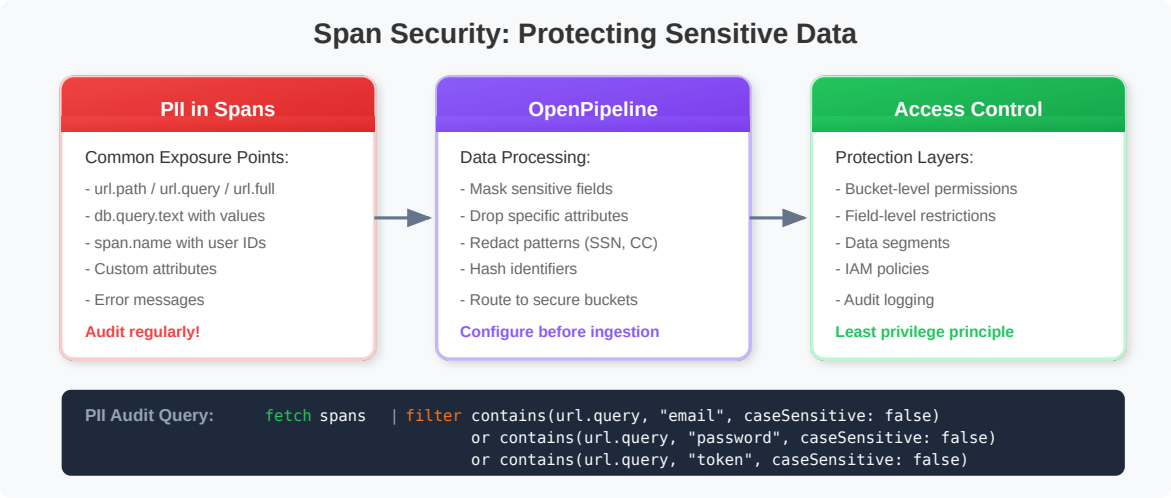
Prerequisites

Before starting this notebook, ensure you have:

- ✅ Completed previous SPANS notebooks (01-05)
- ✅ Understanding of HTTP status codes and security concepts
- ✅ Access to span data containing HTTP attributes (`storage:spans:read`)
- ✅ Familiarity with OpenPipeline basics

1. Understanding Sensitive Data in Spans

Distributed traces can inadvertently capture sensitive information:



Fields to Consider for Protection

Field	Risk	Recommendation
`url.path`	PII in the path	Mask, or group by `http.route`
`url.query` / `url.full`	Tokens and credentials in query parameters	Mask parameter values
`http.request.header.<name>`	Auth tokens/cookies (only headers you configured for capture)	Don't capture them; remove if captured

Field	Risk	Recommendation
`db.query.text` (was `db.statement`)	SQL with user data	Parameterize or mask
`span.events` → `exception.message` / `exception.stack_trace`	Values in exception text	Mask or limit capture
Request attributes / custom attributes	Business data	Evaluate case by case

Exceptions are recorded as **span events**, not span fields: on the validation tenant 6,004 spans in an hour carried an `exception` event with `exception.message` and `exception.stack_trace` inside `span.events`, while a top-level `exception.stacktrace` field has no dictionary row and was set on none.

> **Dictionary:** `url.path`, `url.query`, `url.full`, `db.query.text`, `exception.message`, `http.request.header.__key__` (all `stable`); no row for `exception.stacktrace` or `messaging.payload`, read 10/02/2026 (control: `exception.message` returned a row).

---

2. PII Audit Queries

Regularly audit your span data for potential PII exposure. These queries help identify data that may need masking or removal.

Step 1: Find URLs with Email Addresses

```
// Find URLs that might contain email addresses
// Look for @ symbol or URL-encoded %40
fetch spans, from:-1h
| filter isNotNull(url.path)
| filter contains(url.path, "@") or contains(url.path, "%40")
| fields start_time,
         dt.service.name,
         http.route,
         url.path
| dedup http.route
| limit 20
```

Step 2: Find URLs with User Identifiers

```
// Find URLs with potential user identifiers
// contains() is case-sensitive by default, so match case-insensitively
fetch spans, from:-1h
| filter isNotNull(url.path)
| filter contains(url.path, "user", caseSensitive: false) or
        contains(url.path, "email", caseSensitive: false) or
        contains(url.path, "customer", caseSensitive: false) or
        contains(url.path, "account", caseSensitive: false)
| fields dt.service.name,
         http.route,
         url.path
| dedup http.route
| limit 20
```

Step 3: Find Sensitive Query Parameters

```
// Find URLs that suggest credentials or tokens.
// Query strings are carried in url.query (or url.full), NOT url.path — a
path-only check
// never sees "?token=…". Check all three, case-insensitively.
fetch spans, from:-1h
| fieldsAdd url_text = concat(coalesce(url.path, ""), "?", coalesce(url.query,
""), " ", coalesce(url.full, ""))
| filter contains(url_text, "password", caseSensitive: false)
    or contains(url_text, "token", caseSensitive: false)
    or contains(url_text, "secret", caseSensitive: false)
    or contains(url_text, "apikey", caseSensitive: false)
    or contains(url_text, "auth", caseSensitive: false)
| fields dt.service.name, http.route, url.path, url.query
| dedup http.route
| limit 20
```

Step 4: Audit Database Queries for PII

```
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and every one of them is null on Grail spans. They fail
SILENTLY: a filter on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Find database queries that might contain sensitive data (contains() is
case-sensitive by default, so match case-insensitively)
fetch spans, from:-1h
| filter isNotNull(db.query.text)
| filter contains(db.query.text, "password", caseSensitive: false) or
        contains(db.query.text, "ssn", caseSensitive: false) or
        contains(db.query.text, "credit", caseSensitive: false) or
        contains(db.query.text, "email", caseSensitive: false) or
        contains(db.query.text, "phone", caseSensitive: false)
| fields dt.service.name,
         db.system,
         db.namespace,
         db.query.text
| limit 20
```

Step 5: Check Error Messages for Data Leakage

```
// Audit error messages for potential data leakage
fetch spans, from:-1h
| filter span.status_code == "error"
| filter isNotNull(span.status_message)
| fields dt.service.name,
         span.name,
         span.status_message
| limit 20
```

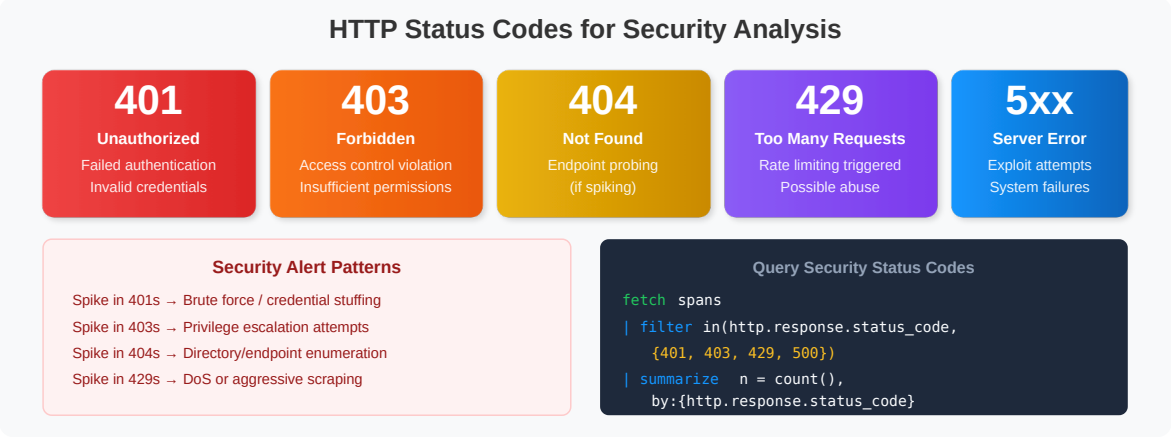
Summary: Which Fields Contain Potentially Sensitive Data?

```
// Identify which potentially sensitive fields are present
fetch spans, from:-1h
| summarize {
    total_spans = count(),
    has_url = countIf(isNotNull(url.path)),
    has_db_statement = countIf(isNotNull(db.query.text)),
    has_exception = countIf(iAny(span.events[][span_event.name] ==
"exception"))
  }
| fieldsAdd url_percent = (has_url * 100.0) / total_spans
| fieldsAdd db_percent = (has_db_statement * 100.0) / total_spans
| fieldsAdd exception_percent = (has_exception * 100.0) / total_spans
```

---

3. HTTP Status Code Security Analysis

HTTP status codes can reveal security-relevant patterns:




```
// Analyze HTTP status code distribution
fetch spans, from:-1h
| filter isNotNull(http.response.status_code)
| summarize {count = count()}, by:{http.response.status_code}
| sort count desc
| limit 20
```

```
// Find security-relevant HTTP errors (401, 403, 429)
fetch spans, from:-1h
| filter in(http.response.status_code, {401, 403, 429})
| summarize {
    count = count()
  }, by:{http.response.status_code, dt.service.name, http.route}
| sort count desc
| limit 50
```

```
// Security status codes over time (detect spikes)
fetch spans, from:-1h
| filter in(http.response.status_code, {401, 403, 429, 500})
| fieldsAdd time_bucket = bin(start_time, 10m)
| fieldsAdd status_category = if(http.response.status_code == 401,
"401_Unauthorized",
                               else: if(http.response.status_code == 403,
"403_Forbidden",
                               else: if(http.response.status_code == 429,
"429_RateLimited",
                               else: "500_ServerError")))
| summarize {count = count()}, by:{time_bucket, status_category}
| sort time_bucket asc, status_category
| limit 200
```

---

4. Authentication Failure Detection

Monitor authentication endpoints for potential brute force or credential stuffing attacks.

```
// Find 401 Unauthorized responses (failed authentication)
fetch spans, from:-1h
| filter http.response.status_code == 401
| fields start_time,
         dt.service.name,
         http.request.method,
         http.route,
         url.path,
         trace.id
| sort start_time desc
| limit 100
```

```
// Count authentication failures by endpoint
fetch spans, from:-1h
| filter http.response.status_code == 401
| summarize {
    failure_count = count(),
    unique_traces = countDistinctExact(trace.id)
  }, by:{dt.service.name, http.route}
| sort failure_count desc
| limit 30
```

```
// Authentication failure rate over time
// contains() is case-sensitive by default, so match case-insensitively
fetch spans, from:-1h
| filter isNotNull(http.response.status_code)
| filter contains(http.route, "auth", caseSensitive: false) or
contains(http.route, "login", caseSensitive: false) or contains(span.name,
"login", caseSensitive: false)
| fieldsAdd time_bucket = bin(start_time, 10m)
| summarize {
    total_attempts = count(),
    failures = countIf(http.response.status_code == 401)
  }, by:{time_bucket}
| fieldsAdd failure_rate_pct = (failures * 100.0) / total_attempts
| sort time_bucket asc
| limit 50
```

```
// High-frequency authentication failures by service
// contains() is case-sensitive by default, so match case-insensitively
fetch spans, from:-1h
| filter contains(span.name, "auth", caseSensitive: false) or
contains(span.name, "login", caseSensitive: false)
| filter span.status_code == "error" or http.response.status_code == 401 or
http.response.status_code == 403
| summarize {
    failure_count = count()
  }, by:{dt.service.name, span.name, http.response.status_code}
| filter failure_count &amp;gt; 10
| sort failure_count desc
```

5. Anomalous Traffic Patterns

Detect unusual patterns that might indicate attacks or misuse.

```
// Find endpoints with unusually high error rates
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(http.response.status_code)
| summarize {
    total_requests = count(),
    error_4xx = countIf(http.response.status_code &gt;= 400 and
http.response.status_code &lt; 500),
    error_5xx = countIf(http.response.status_code &gt;= 500)
  }, by:{dt.service.name, http.route}
| fieldsAdd error_rate_4xx = (error_4xx * 100.0) / total_requests
| fieldsAdd error_rate_5xx = (error_5xx * 100.0) / total_requests
| filter error_rate_4xx &gt; 20 or error_rate_5xx &gt; 5
| sort error_rate_4xx desc
| limit 30
```

```
// Detect potential enumeration attacks (high 404 rates)
fetch spans, from:-1h
| filter http.response.status_code == 404
| summarize {
    not_found_count = count(),
    unique_paths = countDistinct(url.path)
  }, by:{dt.service.name}
| filter not_found_count &gt; 100
| sort not_found_count desc
| limit 20
```

```
// Find rate limiting events (429 responses)
fetch spans, from:-1h
| filter http.response.status_code == 429
| fields start_time,
         dt.service.name,
         http.route,
         trace.id
| sort start_time desc
| limit 50
```

```
// Unusual access patterns - services with high error rates
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    requests = count(),
    unique_operations = countDistinct(span.name),
    error_rate = (countIf(span.status_code == "error" or
http.response.status_code &gt;= 500) * 100.0) / count()
  }, by:{dt.service.name}
| filter error_rate &gt; 20
| sort error_rate desc
```

---

6. Sensitive Endpoint Monitoring

Monitor access patterns to sensitive endpoints like admin panels, configuration APIs, and user data endpoints.

```
// Find access to admin or configuration endpoints
// contains() is case-sensitive by default, so match case-insensitively
fetch spans, from:-1h
| filter span.kind == "server"
| filter contains(url.path, "admin", caseSensitive: false)
      or contains(url.path, "config", caseSensitive: false)
      or contains(url.path, "settings", caseSensitive: false)
      or contains(span.name, "admin", caseSensitive: false)
| fields start_time,
         dt.service.name,
         http.request.method,
         url.path,
         http.response.status_code,
         trace.id
| sort start_time desc
| limit 100
```

```
// Monitor data export or bulk operations
// contains() is case-sensitive by default, so match case-insensitively
fetch spans, from:-1h
| filter span.kind == "server"
| filter contains(url.path, "export", caseSensitive: false)
      or contains(url.path, "download", caseSensitive: false)
      or contains(url.path, "bulk", caseSensitive: false)
| summarize {
    access_count = count(),
    unique_traces = countDistinctExact(trace.id)
  }, by:{dt.service.name, http.route}
| sort access_count desc
| limit 20
```

```
// Identify services that might handle regulated data
// contains() is case-sensitive by default, so match case-insensitively
fetch spans, from:-1h
| filter contains(span.name, "health", caseSensitive: false) or
        contains(span.name, "patient", caseSensitive: false) or
        contains(span.name, "payment", caseSensitive: false) or
        contains(span.name, "card", caseSensitive: false) or
        contains(span.name, "billing", caseSensitive: false)
| summarize {
    span_count = count(),
    unique_operations = countDistinct(span.name)
  }, by:{dt.service.name}
| sort span_count desc
```

---

7. OpenPipeline for Data Masking

Use OpenPipeline to mask sensitive data **before** it is stored in Grail.

Masking with a DQL processor

There is no dedicated masking processor. Masking is a **DQL** processor in the **Processing** stage of the spans pipeline, using `replacePattern` with **DPL** patterns (not regex). One processor can mask several fields:

```
fieldsAdd url.path = replacePattern(url.path, "[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+", "[EMAIL-MASKED]")
| fieldsAdd url.query = replacePattern(url.query, "&lt;&lt;(('token' | 'TOKEN' | 'Token' | 'key' | 'KEY' | 'Key' | 'secret' | 'SECRET' | 'Secret' | 'password' | 'PASSWORD' | 'Password') '=') [^&amp;]+", "[REDACTED]")
| fieldsAdd url.path = replacePattern(url.path, "&lt;&lt;'/' CREDITCARD (&gt;&gt;'/' | &gt;&gt;'?' | EOF)", "[CARD-MASKED]")
```

Each statement was run against sample values on 10/02/2026: `/users/john.doe@example.com/orders` → `/users/[EMAIL-MASKED]/orders`; `token=abc123XYZ&page=2&password=hunter2` → `token=[REDACTED]&page=2&password=[REDACTED]`; `/pay/4111111111111111/confirm` → `/pay/[CARD-MASKED]/confirm`. The card statement is bounded to a whole path segment on purpose: a bare `CREDITCARD` matcher also hits Luhn-valid digit runs **inside** hex identifiers — on the validation tenant it rewrote `/session/c05e365fe5eee852102159234007ec45/element` to `/session/c05e365fe5eee8[CARD]7ec45/element`, and all 30 "card" hits in an hour were session hashes of that kind. `<<(…)` is a lookbehind, so the parameter name stays and only the value is replaced. DPL literals are case-sensitive and there is no case-insensitive flag, so the key list names each casing: with lowercase keys only, `API_KEY=ABCDEF&page=2`, `TOKEN=…` and `Password=…` were left in clear, and with the list above they are masked (verified 10/12/2026). `key` also masks `api_key` and `token` also masks `access_token`, because the lookbehind only checks the characters before `=`. The same reason makes it over-mask `monkey=`, which is the safe direction. `replacePattern` has no back-references, but partial masking ("keep the last four digits") still works with a `>>` lookahead, which checks the trailing digits without consuming them: `replacePattern(url.path, "<<'/' [0-9]{12} >>([0-9]{4} ('/' | '?' | EOF))", "XXXXXXXXXXXX")` turns `/pay/4111111111111234/confirm` into `/pay/XXXXXXXXXXXX1234/confirm` (verified 10/12/2026). Apply the same statements to `url.full` if your spans carry it. For the DPL syntax in depth, see **OPLOGS-08** and **FAQ-15**.

Routing sensitive spans to restricted buckets

Routing chooses a **pipeline**; the pipeline's **Bucket assignment** stage chooses the bucket. Matchers accept `matchesValue` / `matchesPhrase` / comparisons, not `contains()` or `in()`:

Route matcher (first match wins)	Target pipeline	Its Bucket assignment
`matchesValue(dt.service.name, "payment*") or matchesValue(dt.service.name, "auth*")`	`sensitive-spans`	`spans_sensitive`
`matchesValue(deployment.environment, "production")`	`production-spans`	`spans_production`
(default route)	default pipeline	`default_spans`

Spans OpenPipeline configuration is covered in **SPANS-07** and **OPIPE-02**.

Verify Masking Is Working

```
// Verify masking is working — leak check.
// Counts values the masking patterns would STILL change. All three should be
0 after masking.
fetch spans, from:-1h
| summarize {
    total = count(),
    unmasked_email = countIf(replacePattern(url.path, "[A-Za-z0-9._%+-]+ '@'
[A-Za-z0-9.-]+", "") != url.path),
    unmasked_secret_param = countIf(replacePattern(url.query, "&amp;lt;&amp;lt;
(('token' | 'TOKEN' | 'Token' | 'key' | 'KEY' | 'Key' | 'secret' | 'SECRET' |
'Secret' | 'password' | 'PASSWORD' | 'Password') '=') [^&amp;amp;]+", "") !=
url.query),
    unmasked_card = countIf(replacePattern(url.path, "&amp;lt;&amp;lt;'/' CREDITCARD
(&amp;gt;&amp;gt;'/' | &amp;gt;&amp;gt;'?' | EOF)", "") != url.path)
  }
```

---

8. Compliance Considerations

GDPR Compliance

- **Data minimization**: Only collect necessary data
- **Purpose limitation**: Use data only for stated purposes
- **Storage limitation**: Define retention periods

- **Right to erasure**: Plan for data deletion requests

PCI DSS Compliance

- **Do not let** full card numbers reach traces — PCI DSS calls for stored card numbers to be rendered unreadable; confirm the controls that apply to you with your compliance team
- **Mask** cardholder data in all traces
- **Mask** at ingestion rather than relying on storage encryption alone
- **Audit** access to payment-related traces

HIPAA Compliance

- **Protect** PHI (Protected Health Information)
- **Encrypt** health-related span data
- **Restrict** access based on role
- **Audit** all access to healthcare traces

Secure Query Patterns

Use aggregations instead of exposing individual records:

```
// SECURE: Aggregate patterns without exposing individual data
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    request_count = count(),
    unique_routes = countDistinct(http.route),
    error_count = countIf(span.status_code == "error")
  }, by:{dt.service.name}
| sort request_count desc
```

Use `http.route` (pattern) instead of `url.path` (full URL):

```
// SECURE: Use http.route (pattern) instead of url.path (full URL)
// http.route: /users/:id (pattern, no PII)
// url.path: /users/john.doe@email.com (actual value, potential PII)

fetch spans, from:-1h
| filter isNotNull(http.route)
| summarize {
    requests = count(),
    avg_ms = avg(duration) / 1ms
  }, by:{dt.service.name, http.route, http.request.method}
| sort requests desc
| limit 20
```

Limit exposure of trace.id - get just enough to diagnose:

```
// For investigation: Get just enough trace.ids to diagnose
fetch spans, from:-1h
| filter span.status_code == "error"
| summarize {
    error_count = count(),
    sample_trace = takeFirst(trace.id)
  }, by:{dt.service.name, span.name}
| sort error_count desc
| limit 10
```

---

9. Security Audit Queries

Use these queries for regular security audits.

```
// Security summary dashboard: Overall security status
fetch spans, from:-1h
| filter isNotNull(http.response.status_code)
| summarize {
    total_requests = count(),
    auth_failures_401 = countIf(http.response.status_code == 401),
    forbidden_403 = countIf(http.response.status_code == 403),
    rate_limited_429 = countIf(http.response.status_code == 429),
    server_errors_5xx = countIf(http.response.status_code &gt;= 500)
  }
| fieldsAdd auth_failure_rate = (auth_failures_401 * 100.0) / total_requests
| fieldsAdd forbidden_rate = (forbidden_403 * 100.0) / total_requests
```

```
// Security events timeline
fetch spans, from:-24h
| filter in(http.response.status_code, {401, 403, 429})
| makeTimeseries {
    auth_failures = countIf(http.response.status_code == 401),
    forbidden = countIf(http.response.status_code == 403),
    rate_limited = countIf(http.response.status_code == 429)
  }, interval: 10m
```

```
// Services with highest security event rates
fetch spans, from:-1h
| filter isNotNull(http.response.status_code)
| summarize {
    total_requests = count(),
    security_events = countIf(in(http.response.status_code, {401, 403, 429}))
  }, by:{dt.service.name}
| fieldsAdd security_event_rate = (security_events * 100.0) / total_requests
| filter security_events &gt; 0
| sort security_event_rate desc
| limit 20
```

```
// Final audit: Summary of potential security concerns
// contains() is case-sensitive by default, so match case-insensitively
fetch spans, from:-1h
| summarize {
    total_spans = count(),
    spans_with_url = countIf(isNotNull(url.path)),
    spans_with_db_query = countIf(isNotNull(db.query.text)),
    spans_with_errors = countIf(span.status_code == "error"),
    auth_failures = countIf(
        (contains(span.name, "auth", caseSensitive: false) or
contains(span.name, "login", caseSensitive: false)) and
        (span.status_code == "error" or http.response.status_code &gt;= 400))
  }
| fieldsAdd url_percent = (spans_with_url * 100.0) / total_spans
| fieldsAdd db_percent = (spans_with_db_query * 100.0) / total_spans
```

10. Security Checklist

Data Protection

- [ ] Audit spans for PII in URLs, headers, query params
- [ ] Configure OpenPipeline to mask sensitive data
- [ ] Use `http.route` instead of `url.path` when possible
- [ ] Drop or mask database query contents
- [ ] Sanitize error messages and stack traces

Access Control

- [ ] Route sensitive spans to restricted buckets
- [ ] Configure IAM policies for span access
- [ ] Limit who can query raw span data
- [ ] Use aggregations instead of exposing individual records

Compliance

- [ ] Define data retention policies per bucket
- [ ] Document data processing for GDPR Article 30
- [ ] Ensure PCI DSS compliance for payment spans
- [ ] Protect PHI for HIPAA compliance

Monitoring

- [ ] Regular audits for new PII exposure
- [ ] Monitor authentication failure patterns
- [ ] Alert on unusual access patterns
- [ ] Review masking effectiveness periodically

Summary

In this notebook, you learned:

✅ **Understanding sensitive data locations** in span attributes
✅ **PII audit queries** to find emails, tokens, and credentials in spans
✅ **HTTP status code analysis** for security-relevant codes (401, 403, 429, 500)
✅ **Authentication failure detection** to identify potential attacks
✅ **Anomalous traffic pattern detection** for enumeration and abuse
✅ **Sensitive endpoint monitoring** for admin and data access
✅ **OpenPipeline masking** with DQL-processor `replacePattern` statements for emails, card numbers and tokens
✅ **Compliance considerations** for GDPR, PCI DSS, and HIPAA
✅ **Security audit queries** for compliance and reporting
✅ **Security checklist** for ongoing protection

Next Steps

Continue to **SPANS-07: Grail Buckets & OpenPipeline** to learn:
- Understanding Grail bucket architecture
- Configuring OpenPipeline for span processing
- Data routing and retention strategies
- Access control for span data

References

- Trace semantic conventions (DT docs)
- Processing stage (DT docs)
- DQL matcher in OpenPipeline (DT docs)
- Dynatrace Pattern Language (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.